

Report: Intrusion Detection Using Snort

Implementation and Verification of Snort Logging, Custom IDS Rules, Nmap Scan Detection, and DoS Detection

Item	Details
Course	Information Security, 2026 Spring Semester
Student / Client	20102105 Kim Seungjun
Lab Partner / Server	22101997 Park Seonghun
Lab Environment	Docker-based Linux container running Snort; traffic generated from host/client terminal
Main Tools	Snort 2.9.20, tcpdump, curl, Nmap/nping, netcat, Docker, VS Code terminal
Lab Evidence Date	19 May 2026, based on terminal alert timestamps

Formal lab report with execution screenshots, rule analysis, and result interpretation.

Contents and Evidence Checklist

This report is structured as a lab report rather than a command-only README. Each experiment includes the objective, commands, rule logic, evidence screenshot, result interpretation, and learning outcome.

Section	Purpose
1. Introduction and Purpose	Explains the overall goal of the Snort IDS lab.
2. Background	Introduces NIDS, Snort, Nmap, and Snort rule structure.
3. Environment and Network Roles	Documents the actual lab setup and observed IP addresses.
4. Basic Snort Packet Logging	Verifies binary packet logging and tcpdump analysis.
5. HTTP Content Rule Detection	Shows custom content matching for www.seoultech.ac.kr.
6. Port Scan Detection	Detects Nmap reconnaissance traffic to port 1234.
7. DoS/SYN Flood Detection	Detects repeated TCP SYN packets using a threshold rule.
8. Troubleshooting and Limitations	Explains Docker checksum behavior and command differences.
9. Conclusion and Learning Outcomes	Summarizes what was learned from the lab.

Mandatory Screenshot Checklist

Assignment Requirement	Included in This Report	Evidence Figure
Page 48: custom SeoulTech rule alert	Yes	Figure 3
Page 60: scanning detection alert	Yes	Figure 4
Page 83: DoS/DDoS detection alert	Yes	Figure 5
Code execution process and results	Yes	Figures 1-5 and command explanations

1. Introduction and Purpose

The purpose of this lab was to understand how a network intrusion detection system (NIDS) monitors traffic and how Snort can be configured to detect specific events using custom rules. The practice covered three main detection scenarios: HTTP content matching, port-scan detection, and DoS/SYN-flood detection.

The work was performed in a Docker-based Linux environment. Snort was run inside the container and inspected packets on the eth0 interface. The host/client side generated test traffic using curl, Nmap/nping, and netcat. The screenshots included in this report prove that the commands were executed and that Snort generated the expected alerts.

Instead of presenting only commands, this report explains why each command was used, what each Snort rule option means, and how the terminal output verifies the result. This format makes the report understandable to a reader who did not directly perform the lab.

2. Environment and Network Roles

The lab was performed with a client/server model. The client generated test traffic, while the server/container ran Snort and observed the network packets. The actual evidence screenshots show the Docker container prompt as root@2af7b0a26c0e and Snort version 2.9.20 GRE Build 82.

Component	Observed / Used Value	Role
Snort container	root@2af7b0a26c0e	IDS sensor and target environment
Snort version	2.9.20 GRE Build 82	Detection engine used in the screenshots
Snort interface	eth0	Interface monitored by Snort
Docker gateway	172.17.0.1	Gateway visible in ARP log
Snort container IP in alert tests	172.18.0.2	Destination IP in scan and DoS evidence
Traffic source/host IP	192.168.65.1	Client-side source visible in scan and DoS alerts

External SeoulTech HTTP server	203.246.83.174:80	Remote endpoint used for HTTP content rule test
--------------------------------	-------------------	---

Because the environment used Docker, some commands differ from a pure Ubuntu/WSL lab. In particular, checksum validation had to be disabled using the Snort option `-k none` so that Snort would not discard packets affected by Docker checksum offloading before rule matching.

3. Basic Snort Packet Logging and tcpdump Verification

3.1 Objective

The first task verified that Snort could capture traffic and save it as a binary packet log. The generated log was then opened with tcpdump to confirm the captured packet contents. This step demonstrates the difference between Snort packet logging mode and alert mode.

3.2 Commands Used

```
./start.sh
snort -b -n 1
curl localhost:1234
tcpdump -r /var/log/snort/snort.log.1779170771
```

Command / Option	Explanation
<code>./start.sh</code>	Started the prepared Docker-based lab environment.
<code>snort -b -n 1</code>	<code>-b</code> logs packets in binary tcpdump-compatible format; <code>-n 1</code> stops Snort after one packet.
<code>curl localhost:1234</code>	Generated local traffic toward the exposed service/port for the logging test.
<code>tcpdump -r</code>	Read a saved packet log file instead of capturing live traffic.

3.3 Result and Interpretation

The first screenshot shows Snort exiting after capturing one packet. The protocol statistics show `ARP = 1` and `Total = 1`, which means the first observed packet was an ARP request rather than an HTTP or TCP packet. This is expected in a Docker bridge environment because the host or Docker bridge may first need to resolve the container address before TCP traffic follows.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
stellarcloud@stellarcloudui-MacBookAir snort-practice % ./start.sh

Frag: 0 ( 0.000%)
ICMP: 0 ( 0.000%)
UDP: 0 ( 0.000%)
TCP: 0 ( 0.000%)
IP6: 0 ( 0.000%)
IP6 Ext: 0 ( 0.000%)
IP6 Opts: 0 ( 0.000%)
Frag6: 0 ( 0.000%)
ICMP6: 0 ( 0.000%)
UDP6: 0 ( 0.000%)
TCP6: 0 ( 0.000%)
Teredo: 0 ( 0.000%)
ICMP-IP: 0 ( 0.000%)
IP4/IP4: 0 ( 0.000%)
IP4/IP6: 0 ( 0.000%)
IP6/IP4: 0 ( 0.000%)
IP6/IP6: 0 ( 0.000%)
GRE: 0 ( 0.000%)
GRE Eth: 0 ( 0.000%)
GRE VLAN: 0 ( 0.000%)
GRE IP4: 0 ( 0.000%)
GRE IP6: 0 ( 0.000%)
GRE IP6 Ext: 0 ( 0.000%)
GRE PPTP: 0 ( 0.000%)
GRE ARP: 0 ( 0.000%)
GRE IPX: 0 ( 0.000%)
GRE Loop: 0 ( 0.000%)
MPLS: 0 ( 0.000%)
ARP: 1 (100.000%)
IPX: 0 ( 0.000%)
Eth Loop: 0 ( 0.000%)
Eth Disc: 0 ( 0.000%)
IP4 Disc: 0 ( 0.000%)
IP6 Disc: 0 ( 0.000%)
TCP Disc: 0 ( 0.000%)
UDP Disc: 0 ( 0.000%)
ICMP Disc: 0 ( 0.000%)
All Discard: 0 ( 0.000%)
Other: 0 ( 0.000%)
Bad Chk Sum: 0 ( 0.000%)
Bad TTL: 0 ( 0.000%)
S5 G 1: 0 ( 0.000%)
S5 G 2: 0 ( 0.000%)
Total: 1

=====

Memory Statistics for File at:Tue May 19 06:06:37 2026

Total buffers allocated: 0
Total buffers freed: 0
Total buffers released: 0
Total file mempool: 0
Total allocated file mempool: 0
Total freed file mempool: 0
Total released file mempool: 0

Heap Statistics of file:
  Total Statistics:
    Memory in use: 0 bytes
    No of allocs: 0
    No of frees: 0

=====

Snort exiting
root@2af7b0a26c0e:/app#

```

Figure 1. Snort binary logging test: Snort captured one packet and reported ARP = 1, Total = 1.

The second screenshot confirms the saved Snort log with tcpdump. The first command failed because tcpdump interpreted the file path as a capture filter. The corrected command used -r, which tells tcpdump to read from the saved log file. The log output shows an ARP request: "who-has 2af7b0a26c0e tell 172.17.0.1, length 28". This proves that Snort saved a real network packet and that the log can be analyzed after capture.

```

root@2af7b0a26c0e:/app# tcpdump /var/log/snort/snort.log.1779170771
tcpdump: can't parse filter expression: syntax error
root@2af7b0a26c0e:/app# tcpdump -r /var/log/snort/snort.log.1779170771
reading from file /var/log/snort/snort.log.1779170771, link-type EN10MB (Ethernet), snapshot length 1514
06:06:37.390429 ARP, Request who-has 2af7b0a26c0e tell 172.17.0.1, length 28
root@2af7b0a26c0e:/app#

```

Figure 2. tcpdump verification of the Snort binary log using the corrected -r option.

Result: The logging function worked successfully. The captured traffic was ARP traffic created by the Docker network, and the saved binary log was readable with tcpdump.

4. HTTP Content-Based Custom Rule Detection

4.1 Objective

The second task created a custom Snort rule that detects TCP packets containing the string `www.seoultech.ac.kr`. This demonstrates content-based signature detection, where Snort inspects packet payloads for a specific pattern and prints an alert when the pattern appears.

4.2 Rule and Configuration

```
alert tcp any any -> any any (content:"www.seoultech.ac.kr"; msg:"SEOULTECH is opened";  
sid:123123;)
```

Field	Meaning
alert tcp	Generate an alert for matching TCP traffic.
any any -> any any	Apply the rule to traffic from any source IP/port to any destination IP/port.
content:"www.seoultech.ac.kr"	Look for the SeoulTech domain string in the packet payload.
msg:"SEOULTECH is opened"	Display this message when the rule matches.
sid:123123	Unique rule identifier for this custom rule.

The custom rules file was included in `snort.conf` with the following line so that Snort would load it during startup.

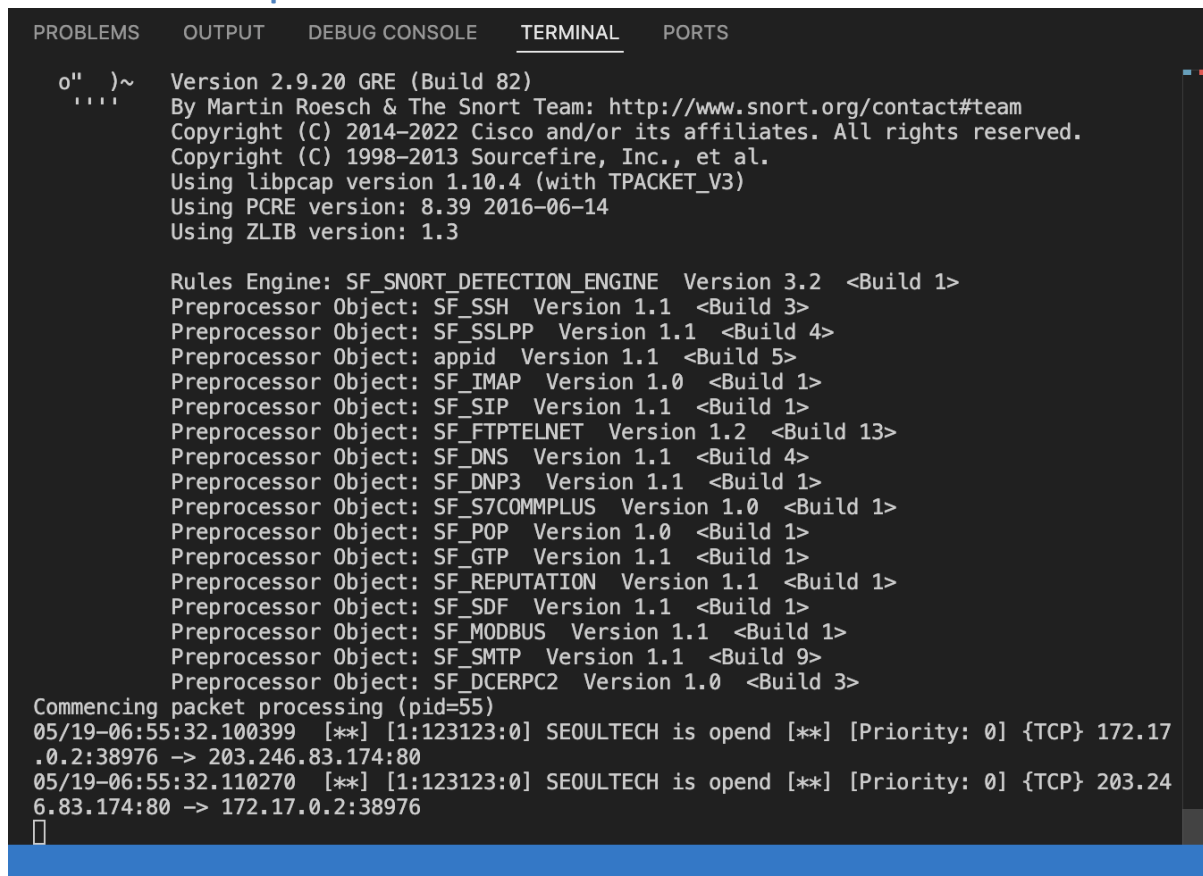
```
include $RULE_PATH/snort.rules
```

4.3 Execution

```
snort -c /etc/snort/snort.conf -A console -i eth0 -k none  
curl -v http://www.seoultech.ac.kr
```

The `-c` option loads the Snort configuration file, `-A console` prints alerts directly in the terminal, `-i eth0` selects the interface to monitor, and `-k none` disables checksum validation for the Docker environment. The `curl` command uses the `http://` scheme so the HTTP request is not encrypted and the domain string can be inspected by Snort. The `-v` option only enables verbose curl output; the use of plain HTTP is what makes content inspection possible.

4.4 Evidence and Interpretation



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

o" }~ Version 2.9.20 GRE (Build 82)
' ' ' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
      Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
      Copyright (C) 1998-2013 Sourcefire, Inc., et al.
      Using libpcap version 1.10.4 (with TPACKET_V3)
      Using PCRE version: 8.39 2016-06-14
      Using ZLIB version: 1.3

      Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
      Preprocessor Object: SF_SSH Version 1.1 <Build 3>
      Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
      Preprocessor Object: appid Version 1.1 <Build 5>
      Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
      Preprocessor Object: SF_SIP Version 1.1 <Build 1>
      Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
      Preprocessor Object: SF_DNS Version 1.1 <Build 4>
      Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
      Preprocessor Object: SF_S7COMPLUS Version 1.0 <Build 1>
      Preprocessor Object: SF_POP Version 1.0 <Build 1>
      Preprocessor Object: SF_GTP Version 1.1 <Build 1>
      Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
      Preprocessor Object: SF_SDF Version 1.1 <Build 1>
      Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
      Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
      Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Commencing packet processing (pid=55)
05/19-06:55:32.100399 [**] [1:123123:0] SEOULTECH is opend [**] [Priority: 0] {TCP} 172.17
.0.2:38976 -> 203.246.83.174:80
05/19-06:55:32.110270 [**] [1:123123:0] SEOULTECH is opend [**] [Priority: 0] {TCP} 203.24
6.83.174:80 -> 172.17.0.2:38976
█
```

Figure 3. Mandatory evidence for page 48 requirement: Snort detected the SeoulTech HTTP content rule.

The alert output shows the custom message "SEOULTECH is opend" with sid 123123. The first alert shows traffic from 172.17.0.2:38976 to 203.246.83.174:80, and the second shows the reverse direction. This proves that the custom rule was correctly loaded and that Snort matched the SeoulTech domain string in TCP/HTTP traffic.

5. Port Scan Detection Using a Local Rule

5.1 Objective

The third task detected reconnaissance behavior by writing a rule for TCP traffic destined to port 1234 on the Snort target. Port scanning is commonly performed before an attack to identify reachable services. In this lab, Nmap was used to generate controlled scan traffic and Snort was used to detect it.

5.2 Rule and Configuration

```
alert tcp any any -> 172.18.0.2 1234 (msg:"Scanning_tmp1"; flow:stateless;
classtype:attempted-recon; sid:13;)
```

Rule Element	Explanation
172.18.0.2 1234	The Snort/container target IP and the scanned port. This value must be changed to the current server IP in another environment.
msg:"Scanning_tmp1"	Alert message printed when scanning traffic is detected.
flow:stateless	Allows detection even if Snort does not see or track a full TCP session. This is useful for scan packets.
classtype:attempted-recon	Classifies the alert as reconnaissance / attempted information gathering.
sid:13	Custom rule ID used for this scan detection rule.

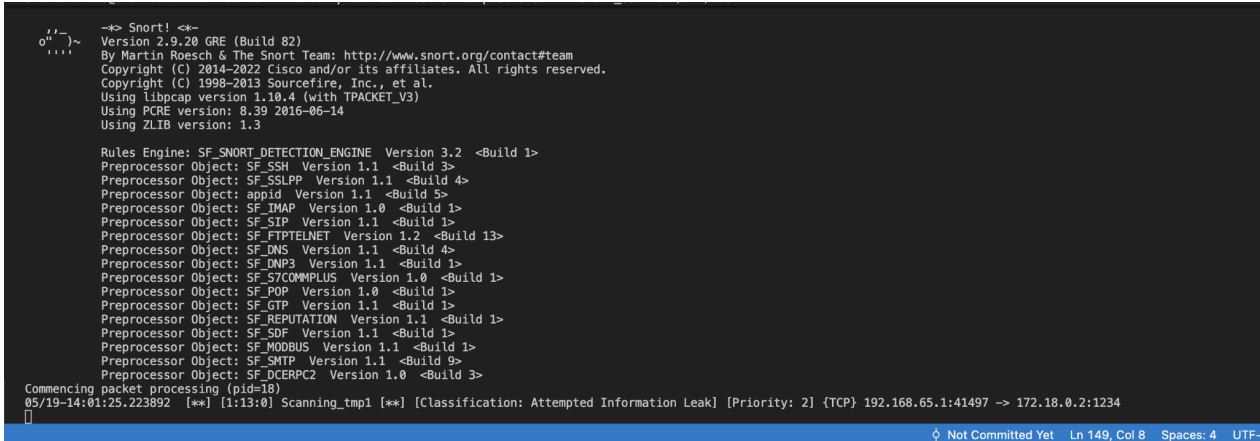
The rule was activated by including the local rules file in snort.conf. In a non-Docker lab, netcat can be used to open port 1234 before scanning. In this Docker-based test, the container setup already exposed the required port, so an additional netcat listener was not necessary.

5.3 Execution

```
snort -c /etc/snort/snort.conf -A console -i eth0 -k none
nmap -sT -p 1234 172.18.0.2
```

A TCP connect scan (-sT) was used in the Docker/host environment. In a direct WSL or native Linux environment, a SYN scan (-sS) may be used when raw socket privileges are available. Both methods generate TCP traffic toward the target port and can trigger the Snort rule.

5.4 Evidence and Interpretation



```
--> Snort! <--
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_S7COMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>

Commencing packet processing (pid=18)
05/19-14:01:25.223892 [**] [1:13:0] Scanning_tmp1 [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.65.1:41497 -> 172.18.0.2:1234
```

Figure 4. Mandatory evidence for page 60 requirement: Snort detected Nmap scanning traffic to port 1234.

The alert line shows "Scanning_tmp1" with classification "Attempted Information Leak" and priority 2. The packet direction is 192.168.65.1:41497 -> 172.18.0.2:1234, which means the host/client generated traffic toward the Snort target port. Therefore, the custom rule successfully detected the scan attempt.

6. DoS/SYN Flood Detection

6.1 Objective

The fourth task detected a DoS-like pattern by watching for repeated TCP SYN packets. A SYN packet starts a TCP connection. If many SYN packets are sent quickly to the same destination, the behavior can resemble a SYN flood, which may exhaust server resources or indicate an attack attempt.

6.2 Rule and Threshold Logic

```
alert tcp any any -> any any (msg:"DOS ATTACK IS DETECTED"; flags:S; threshold: type
threshold, track by_dst, count 20, seconds 60; sid:5000;)
```

Option	Explanation
flags:S	Match packets with the TCP SYN flag set. These packets indicate connection attempts.
threshold: type threshold	Suppress normal repeated alerts and alert only when the threshold is reached.
track by_dst	Count matching packets by destination IP address.
count 20, seconds 60	Trigger when the count reaches 20 matching events within 60 seconds.
sid:5000	Custom rule ID for the DoS/SYN-flood detection rule.

6.3 Execution

```
snort -c snort.conf -A console -i eth0 -k none

# Windows / Nmap nping example
nping --tcp --flags SYN -p 80 --rate 5 --count 20 172.18.0.2

# Mac host alternative used for parallel connection attempts
seq 60 | xargs -I {} -P 60 nc -zv -G 1 172.18.0.2 1234
```

The nping command sends TCP SYN packets at a controlled rate and count. The Mac alternative starts multiple parallel netcat connection attempts, which also produces many TCP connection attempts toward the target. In both cases, the rule detects the repeated SYN-like traffic pattern by counting events per destination.

6.4 Evidence and Interpretation

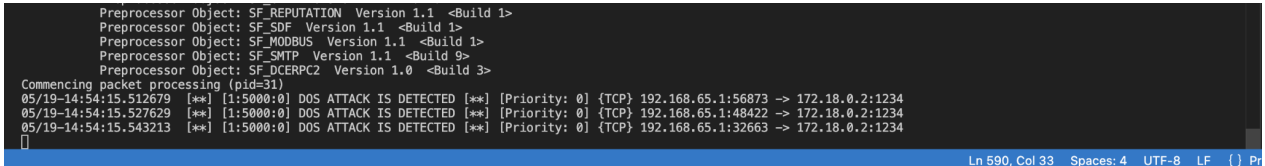
A terminal window showing Snort logs. At the top, several preprocessors are listed: SF_REPUTATION, SF_SDF, SF_MODBUS, SF_SMTP, and SF_DCERPC2. Below these, a message says 'Commencing packet processing (pid=31)'. Then, three lines of alerts are shown, each starting with a timestamp (05/19-14:54:15), a SID (1:5000:0), and the message 'DOS ATTACK IS DETECTED'. Each alert also includes a priority of 0 and a rule description: '[TCP] 192.168.65.1:56873 -> 172.18.0.2:1234'. The source IP is 192.168.65.1 and the destination is 172.18.0.2:1234. The status bar at the bottom indicates 'Ln 590, Col 33 Spaces: 4 UTF-8 LF () Pr'.

Figure 5. Mandatory evidence for page 83 requirement: Snort detected repeated DoS/SYN-like traffic.

The screenshot shows repeated alerts with the message "DOS ATTACK IS DETECTED" and sid 5000. The source is 192.168.65.1 with different ephemeral ports, and the destination is 172.18.0.2:1234. Multiple alerts indicate that the number of matching TCP SYN connection attempts exceeded the configured threshold during the test. This confirms that Snort was able to detect the DoS/SYN-flood simulation.

7. Troubleshooting and Limitations

7.1 tcpdump File Reading Error

During log verification, running tcpdump followed directly by the log file path caused a syntax error because tcpdump treated the path as a live capture filter expression. The correct method is to use tcpdump -r followed by the saved file path. After adding -r, tcpdump successfully read the Snort binary log.

7.2 Docker Checksum Offloading

Inside Docker, packet checksums may appear invalid to Snort because the actual checksum calculation can be deferred to the physical network interface. If Snort validates those checksums before the final NIC writes them, it may discard the packets before rule matching. The workaround used in this lab was to add -k none to the Snort command. Another possible solution is to disable TX checksum offloading with ethtool in environments where that is allowed.

```
snort -c /etc/snort/snort.conf -A console -i eth0 -k none

# Alternative kernel-level approach, if permitted
apt-get install -y ethtool
ethtool -K eth0 tx off
```

7.3 Nmap Mode Difference

The original lab describes SYN scanning, but Docker environments may restrict raw socket behavior depending on configuration. For that reason, the Docker test used TCP connect scanning (-sT) for the port-scan experiment. The rule still detected the scan because the important condition was TCP traffic reaching port 1234 on the target.

7.4 Rule Scope Limitation

The custom rules used in this lab are intentionally simple for educational purposes. A production IDS rule should usually include tighter source/destination scope, a revision number (rev), and more precise content or flow conditions to reduce false positives. However, these simple rules are appropriate for demonstrating how Snort rule matching works in a controlled lab.

8. Conclusion and Learning Outcomes

This practice showed how Snort can be used as both a packet logger and a rule-based intrusion detection engine. In packet logging mode, Snort saved a binary log that was later verified with tcpdump. In alert mode, Snort successfully detected HTTP content matching, Nmap-style scanning, and repeated SYN/connection-attempt traffic.

The most important lesson was that writing a rule is only one part of IDS operation. The environment, interface, checksum validation, traffic direction, and target IP address must all be correct before Snort can generate meaningful alerts. The Docker checksum issue was especially important because traffic could pass through the interface but still be ignored before rule matching if checksum validation was not adjusted.

Through this lab, I learned how to interpret Snort alert fields, how to connect rule syntax with network behavior, and how to verify IDS results using evidence screenshots. The included screenshots demonstrate that the required lab steps were performed and that the custom rules produced the expected detection results.

Key Learning Outcomes

- **Snort operation modes:** Snort can be used as a packet logger and as a rule-based alerting engine depending on the selected options.
- **Rule-based detection:** Custom rules can detect payload strings, port-scan behavior, and repeated SYN packets by combining rule headers and options.
- **Alert interpretation:** The source IP, destination IP, port, sid, priority, and classification fields in the alert output are essential for verifying what was detected.
- **Environment awareness:** Docker networking and checksum offloading can affect IDS behavior, so environment-specific troubleshooting is part of the detection process.

References

- Course lab handout: Intrusion detection using Snort, Information Security, 2026 Spring Semester.
- Original practice README and execution screenshots from the Snort practice environment.
- Snort terminal output and tcpdump logs captured during the lab on 19 May 2026.